

For a **Generative AI project assignment** aimed at hiring candidates with **AI/ML experience**, especially in **Hugging Face, PyTorch, LangChain, NLP, and Neural Networks**, the task should assess:

1. **Hands-on Hugging Face model usage & fine-tuning**
2. **LangChain for LLM-based applications**
3. **Building & optimizing NLP pipelines**
4. **Neural network knowledge** (Transformer-based models)
5. **Problem-solving skills in AI/ML projects**

Assignment Task: Build a Custom AI-Powered Q&A System

Task Description:

You are required to build a **Question-Answering (QA) chatbot** using a **Hugging Face transformer model** and integrate it with **LangChain**. The chatbot should be able to:

- Load and fine-tune a transformer model on **custom data**
- Perform **context-aware Q&A** using **retrieval-augmented generation (RAG)**
- Test an API using **FastAPI or Flask**
- Optimize the pipeline for **low latency & efficiency**

Expected Deliverables

Code Repository (GitHub/Google Drive link)

Model Training & Fine-Tuning Notebook (Jupyter/Colab)

Inference API (FastAPI/Flask)

README.md (Instructions & Architecture Explanation)

Technical Requirements:

1 Data Preparation & Fine-Tuning (Hugging Face + PyTorch)

- Choose a **pre-trained transformer model** (e.g., bert-base-uncased, llama, mistral, any one with 1 billion or around it based on your hardware.)
- Fine-tune the model on a **custom dataset** (e.g., medical, legal, finance, or any domain-specific Q&A, any dataset find in internet.)
- Use **PyTorch Lightning** for training efficiency or any.
- Store the **fine-tuned model** in Hugging Face's **Model Hub**, **make it private and share same model with us in zip format.**

Evaluation: Proper data pre-processing, fine-tuning techniques, and model performance on validation data.

2 RAG-based Q&A with LangChain

- Implement **retrieval-based augmentation (RAG)**
- Use **FAISS** or **ChromaDB** for storing document embeddings
- Use **sentence-transformers** for embedding generation
- Design a **prompt engineering strategy** for effective answers

Evaluation: Understanding of embeddings, retrieval mechanisms, and prompt tuning.

3 Model Deployment via API (FastAPI/Flask)

- Wrap the model into an **inference API**
- Implement **async processing** for speed optimization
- Expose an endpoint /ask that takes **user queries** and returns **generated responses**

Evaluation: API design, optimization, and real-time response generation.

4 Model Optimization & Performance Monitoring

- Implement **quantization** (e.g., **bitsandbytes**, **ONNX**, or **TensorRT**)
- Log model performance using **Weights & Biases** / **TensorBoard**
- Profile inference speed and optimize for **low latency**

Evaluation: Efficiency of inference and optimizations applied.

Bonus Tasks (Optional, for Extra Points!)

UI for Q&A Chatbot

Multi-modal Support (Text + Image)

Evaluation Metrics

Time Duration: 2 Days

1. **Day 1:** Model fine-tuning and dataset preparation & LangChain integration
2. **Day 2:** RAG implementation and API deployment & performance optimization

Evaluation Criteria:

Criteria	Weight (%)
Model fine-tuning & training	30%
LangChain integration (RAG)	20%
API development & deployment	20%
Optimization & Performance	20%
Code quality & documentation	10%

Here's a **code** along with a **sample dataset** structure for the assignment.

Code: Generative AI Q&A System

This includes:

1. **Dataset Preparation** (Custom Q&A data)
2. **Model Fine-Tuning** (Hugging Face + PyTorch)
3. **RAG Implementation** (LangChain + FAISS)
4. **FastAPI for Model Inference**
5. **Basic Streamlit UI**

Folder Structure

```
GenAI_QA_Assignment/  
├── data/  
│   ├── qa_dataset.json  # Custom Q&A dataset  
├── model/  
│   ├── fine_tune.py  # Model fine-tuning script  
│   └── run_inference.py  # Load & test fine-tuned model  
├── api/  
│   └── app.py  # FastAPI backend  
├── ui/  
│   └── chatbot.py  # Streamlit frontend  
├── requirements.txt  # Dependencies  
└── README.md  # Instructions
```

Sample Q&A Dataset (data/qa_dataset.json)

This dataset contains **questions, context, and answers**, suitable for **fine-tuning**:

```
[
  {
    "context": "Hugging Face provides pre-trained models and APIs for NLP tasks.",
    "question": "What does Hugging Face provide?",
    "answer": "Pre-trained models and APIs for NLP tasks."
  },
  {
    "context": "LangChain is an open-source library for chaining LLMs together.",
    "question": "What is LangChain used for?",
    "answer": "Chaining LLMs for advanced AI applications."
  }
]
```

Step 1: Model Fine-Tuning (model/fine_tune.py)

This script fine-tunes bert-base-uncased on the dataset.

```
import json
import torch
from transformers import BertTokenizer, BertForQuestionAnswering,
Trainer, TrainingArguments
from datasets import Dataset

# Load dataset
with open("../data/qa_dataset.json", "r") as f:
    data = json.load(f)

# Prepare dataset
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```

def encode_data(example):
    inputs = tokenizer(example["context"], example["question"],
truncation=True, padding="max_length", max_length=512)
    inputs["labels"] = tokenizer(example["answer"], truncation=True,
padding="max_length", max_length=128)["input_ids"]
    return inputs

dataset = Dataset.from_list(data).map(encode_data)

# Model & Training
model = BertForQuestionAnswering.from_pretrained("bert-base-uncased")
training_args = TrainingArguments(output_dir="fine_tuned_model",
per_device_train_batch_size=8, num_train_epochs=3)

trainer = Trainer(model=model, args=training_args,
train_dataset=dataset)
trainer.train()

# Save model
model.save_pretrained("../model/fine_tuned/")
tokenizer.save_pretrained("../model/fine_tuned/")

```

Step 2: LangChain RAG Implementation (model/run_inference.py)

This loads the fine-tuned model & performs RAG-based retrieval.

```

from transformers import BertTokenizer, BertForQuestionAnswering
import torch
import faiss
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

# Load model
model_path = "../model/fine_tuned/"
tokenizer = BertTokenizer.from_pretrained(model_path)

```

```

model = BertForQuestionAnswering.from_pretrained(model_path)

# Load FAISS DB
embeddings = HuggingFaceEmbeddings(model_name="sentence-
transformers/all-mpnet-base-v2")
db = FAISS.load_local("faiss_index", embeddings)

def answer_question(question):
    docs = db.similarity_search(question, k=3)
    context = " ".join([doc.page_content for doc in docs])

    inputs = tokenizer(context, question, return_tensors="pt",
truncation=True)
    outputs = model(**inputs)
    answer = tokenizer.decode(outputs.start_logits.argmax(),
outputs.end_logits.argmax())

    return answer

# Example
print(answer_question("What is Hugging Face?"))

```

Step 3: API Deployment (api/app.py)

A **FastAPI** app for serving the model via an API.

```

from fastapi import FastAPI
from pydantic import BaseModel
from model.run_inference import answer_question

app = FastAPI()

class QuestionRequest(BaseModel):
    question: str

@app.post("/ask")

```

```
def get_answer(request: QuestionRequest):  
    response = answer_question(request.question)  
    return {"answer": response}
```

Run the API:

```
uvicorn api.app:app --reload --host 0.0.0.0 --port 3750
```

Step 4: Simple UI for Chatbot (ui/chatbot.py)

A **Streamlit** chatbot UI for testing the Q&A system.

```
import streamlit as st  
import requests  
  
st.title("🤖 GenAI Q&A Chatbot")  
question = st.text_input("Ask a question:")  
if st.button("Get Answer"):  
    response = requests.post("http://127.0.0.1:3750/ask",  
json={"question": question}).json()  
    st.write("Answer:", response["answer"])
```

Run the UI:

```
streamlit run ui/chatbot.py
```

Installation & Setup

```
pip install -r requirements.txt
```

requirements.txt:

torch
transformers
datasets
fastapi
uvicorn
faiss-cpu
langchain
streamlit
requests

Expected Outputs

- **Fine-Tuned Model:**  Candidate's Task
- **LangChain RAG Retrieval:**  Candidate's Task
- **API & Frontend Integration:**  Candidate's Task
- **Low Latency Optimization:**  Candidate's Task

Note:-

- 1 - We are looking for your logical and thinking capability with your task completion process and approach.
- 2 - Please Submit this assignment where there you have completed all the parts of assignment or not.